Software Testing Training

By Grace Olagoke

INTRODUCTION

Welcome to software testing tutorial. This course is for anyone aspiring to be a software quality assurance tester. It covers both manual and automation testing along with some practical.

RECCOMENDATION

I will recommend the following materials for personal use:

- Software Testing: An ISTQB-BCS Certified Tester
- TMAP – Quality for Cross Functional Teams

INSTALLATIONS AND TOOLS

- Intellij Idea – for UI test automation
- Postman – for API testing
- GIT – for continuous development
- JIRA – Project Management

DEFINITIONS

- *Software testing is the systematic and methodical examination of a work product using a variety of techniques, with the express intention of attempting to show that it does not fulfil its desired or intended purpose. This is undertaken in an environment that represents most nearly that which will be used in live operation* – ISTQB
- *Software testing is the act of checking the whether software meets it's intended objectives and satisfies expectations.* – Wikipedia
- *Software testing is the process of evaluating and verifying that a software application works as intended, meets user and technical requirements and is free of defects. It ensures quality, reliability and performance before the software is released* – Geeks for Geeks

EXAMPLES OF SOFTWARE FAILURES

1. A data breach at T-mobile led to the confidential records of 50 million customers being compromised. This include sensitive data such as phone numbers and bank accounts details https://www.security.org/identity-theft/breach/t-mobile/
2. On 3rd of May, 2021, TikTok users found they had zero followers of their accounts. https://www.independent.co.uk/tech/why-is-tiktok-not-working-b1879313.html
3. When the UK Government introduced online filing of tax returns, a user could sometimes see the amount that a previous user earned, regardless of the physical location of the two applicants. http://news.bbc.co.uk/1/hi/business/2016945.stm#:~:text=The%20security%20breach%20could%20set,bank%20details%20of%20other%20customers.

IMPORTANCE OF SOFTWARE TESTING

Software testing is important because:

- It protects product quality
- Reduces costs
- Prevents failure
- Ensures that users get reliable and secure experience

Software testing is not just a “nice-to-have”. It is a core engineering practice that keeps software safe, competitive, and trustworthy.

IMPORTANCE OF SOFTWARE TESTING

- **It identifies bugs early** – saves money
- **It improves security** – uncovers vulnerabilities before attackers does, protects user's sensitive data and reduces the risks of breaches, legal issues and reputation damage
- **It ensures high product quality** – verifies that software meets requirement and behaves as intended, confirms compatibility across devices, browsers, and operating systems. It also helps delivers a smooth user experience.
- **It boosts user-satisfaction** – users usually abandon apps that crashes easily, glitches or behaves unpredictably.

EFFECT OF AN ERROR

ERROR

DEFECT

FAILURE

IMPACT OF SOFTWARE FAILURE

- Incorrect software can cause damage to human life (it can cause aircraft to crash in which people can die, or cause a hospital life support to fail)
- It can cause reputational damage to companies, in which people no longer trust the business, and it may result in huge pay outs.
- It can harm the environment, by releasing chemicals or radiation into the atmosphere.

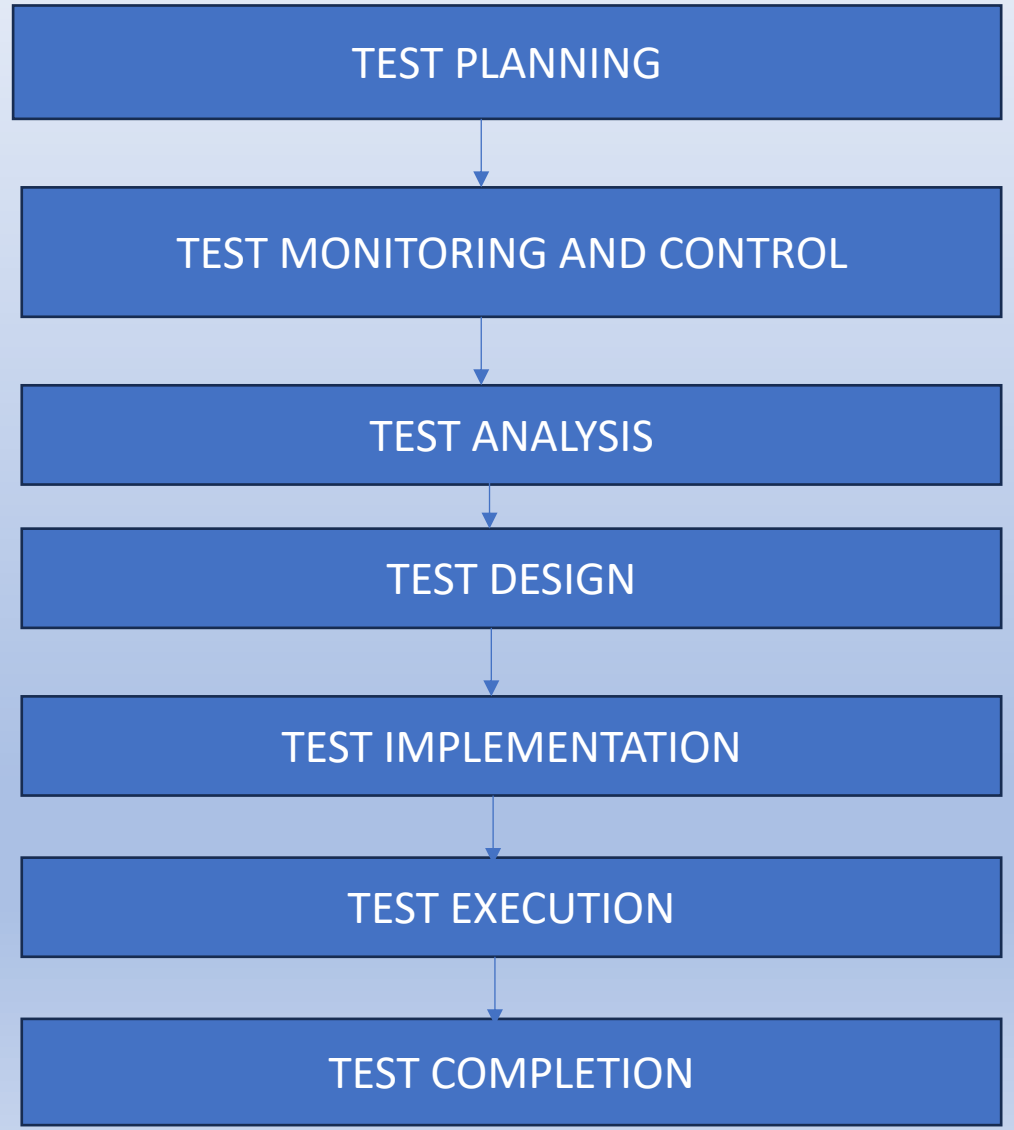
PRINCIPLES OF TESTING

- Exhaustive testing is impossible
- Testing shows the presence of defects, not the absence of it
- Early testing is essential – it saves time and money
- A small number of modules usually contain most defects – defect clustering
- Pesticide paradox – the same test will not uncover new defects
- Testing is context dependent
- The absence-of-error fallacy – just because you did not find any defect does not mean there is none.

TEST ACTIVITIES AND TASK

- Test planning
- Test monitoring and control
- Test analysis – using requirement doc, use cases, user stories etc.
- Test design
- Test implementation
- Test execution
- Test completion

TEST PROCESS



ESSENTIAL SKILLS AND GOOD PRACTICES IN TESTING

- Testing knowledge
- Attention to details and working in methodical way
- Good communication skills, being an active listener and a team player
- Analytical thinking and a creative approach
- Technical knowledge
- Domain knowledge – what the business is involved in, and how the software will help the business achieve their goals.

ESSENTIAL SKILLS AND GOOD PRACTICES IN TESTING

- Whole team approach – everyone is responsible for quality. Testes can educate other members of the team about testing and share their knowledge.
- Independence of testing – testing is more effective if it is not undertaken by the person who wrote the code.

SOFTWARE DEVELOPMENT MODELS

- Waterfall model
- V-model
- Iterative and incremental model

WATERFALL MODEL

Requirement Specification

Functional Specification

Technical Specification

Program Specification

Coding → Test

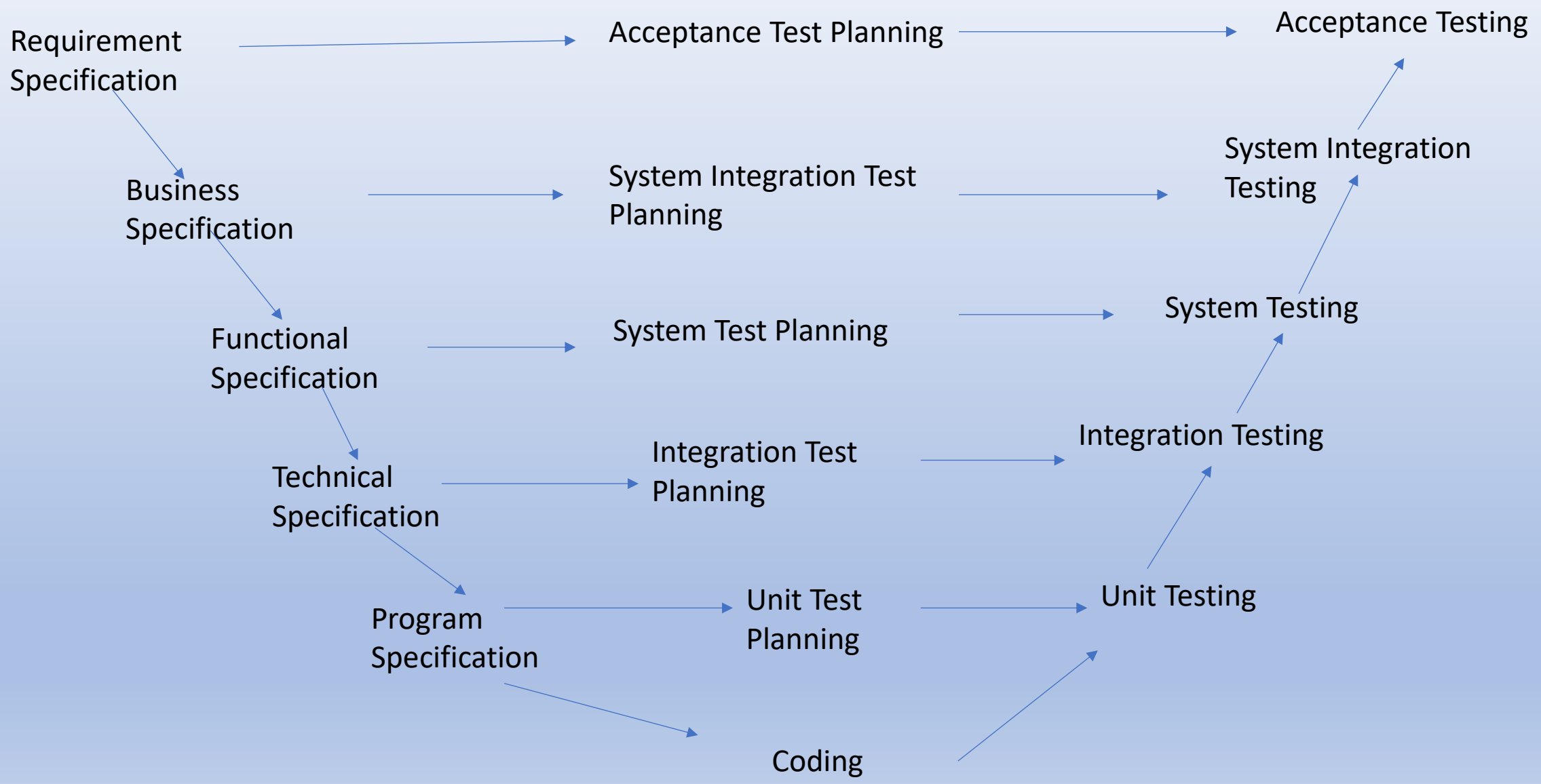
CHARACTERISTICS OF WATERFALL MODEL

- **Linear Progression**: Each phase must be completed before the next one starts, with no overlap.
- **Documentation**: Extensive documentation is created at each stage, which helps in tracking progress and understanding project requirements.
- **Predictability**: The model is best suited for projects with fixed requirements, allowing for accurate time and resource estimates.

DISADVANTAGE OF WATERFALL MODEL

- **Inflexibility**: Difficult to accommodate changes once a phase is completed.
- **Long Development Cycles**: Delays in one phase can impact the entire project timeline.
- **Limited User Involvement**: User feedback is typically gathered only at the beginning and end of the process, which may lead to misalignment with user needs.

V MODEL


Requirement Specification
Acceptance Test Planning
Acceptance Testing
Business Specification
System Integration Test Planning
System Integration Testing
Functional Specification
System Test Planning
System Testing
Technical Specification
Integration Test Planning
Integration Testing
Program Specification
Unit Test Planning
Unit Testing
Coding

ADVANTAGES OF V-MODEL

- It is easy to manage due to the rigidity of the model. Each phase of V-Model has specific deliverables and a review process.
- Proactive defect tracking – that is defects are found at an early stage.

DISADVANTAGES OF V-MODEL

- The V-Model is a linear and sequential model, which can make it challenging to adapt to changing requirements or unforeseen events.
- The V-Model can be time-consuming, as it requires a lot of documentation and testing.
- High risk and uncertainty.
- It is not good for complex and object-oriented projects.
- It is not suitable for projects where requirements are not clear and contain a high risk of changing.
- This model does not support iteration of phases.
- The V-Model places a strong emphasis on documentation, which can lead to an overreliance on documentation at the expense of actual development work.

ITERATIVE DEVELOPMENT MODEL

CHARACTERISTICS OF AGILE MODEL

- The Agile process is all about being flexible, working together, and focusing on delivering real value to customers. Here is the **Characteristics of the Agile Process**:
- Agile processes must be adaptable to technical and environmental changes. That means if any technological changes occur, then the agile process must accommodate them.
- The development of agile processes must be incremental. That means, in each development, the increment should contain some functionality that can be tested and verified by the customer.
- The customer feedback must be used to create the next increment of the process.
- The software increment must be delivered in a short span of time.
- It must be iterative so that each increment can be evaluated regularly.

ADVANTAGES OF AGILE

- Working through Pair programming produces well-written compact programs which have fewer errors as compared to programmers working alone.
- It reduces the total development time of the whole project.
- Agile development emphasizes face-to-face communication among team members, leading to better collaboration and understanding of project goals.
- Customer representatives get the idea of updated software products after each iteration. So, it is easy for him to change any requirement if needed.
- Agile development puts the customer at the center of the development process, ensuring that the end product meets their needs.

DISADVANTAGES OF AGILE

- Lack of formal documentation can create confusion, and important decisions may be misinterpreted by team members.
- Not suitable for projects with complex dependencies.
- Highly dependent on customer interaction; unclear requirements can lead the team in the wrong direction.
- Short development cycles make it difficult to plan, estimate timelines, and manage resources effectively.
- Requires experienced and skilled team members to handle changing requirements and iterative development.
- Limited documentation can make maintenance difficult after project completion, especially when team members move to other projects.

TYPES OF TESTING

- Functional testing
- Non-functional testing
- White box testing
- Black box testing
- Automation testing
- Manual testing
- Specialized testing

Functional Testing

Focuses on what the system does

- Unit testing – Testing individual functions and component
- Integration testing – Ensuring models work together
- System testing – Testing the complete application
- Smoke testing – Quick check to see if the build is stable
- Sanity testing – Verifies specific functionalities after minor changes
- Regression testing – Ensure new changes don’t break existing features
- Acceptance testing – Validate the system with end users

Non-Functional Testing

Focuses on how well the system performs

- Performance testing – speed, responsiveness, stability
- Load testing – Behaviour under expected
- Stress testing – Behaviour under extreme load
- Security testing – Vulnerability, authentication, authorization
- Usability testing – User-friendliness
- Compatibility testing – Browsers, devices, OS
- Reliability testing – Consistency over time

White Box Testing

Tester knows the internal code

- Statement coverage
- Branch coverage
- Path coverage
- Unit test (developer focused)

Black Box Testing

Tester does not know the internal code

- Equivalence partitioning
- Boundary value analysis
- Decision table testing
- State transitioning testing

Manual Testing

- Human driven
- Exploratory, usability, ad-hoc etc.

Automation Testing

- Using tools like selenium, playwright, Robot framework, pytest
- Great for regression testing and for carrying out repetitive tasks
- Also great for CI/CD – continuous integration / continuous deployment

Specialized Testing

- API testing – Postman, RestAssured, Playwright API
- Mobile testing – Appium, Espresso
- Accessibility testing – WCAG compliance
- Database testing – Data integrity, queries
- Recovery testing – system recovery after failure
- Installation and uninstallation testing

Test Cases Generation With Equivalence Partitioning

1. Identify the input condition, for example,

- age range 0-17, 18-65, >65
- Fixed length – 6-digit OTP
- Boolean – Yes or No

2. Determine the valid and invalid partition
3. Select one representative for each partition
4. Write test cases

Example of Equivalent Partitioning

- Input condition - Age field – valid range is 18-56
- Create Partitions

Valid partition – 18-56

Invalid partition - <= 17

Invalid partition - >= 57

- Pick representative values

Valid – 30

Invalid – 10

Invalid - 70

Equivalence Partitioning

Test Case	Input	Partition	Expected Result
TC1	30	valid	Accepted
TC2	10	invalid	Error message
TC3	70	invalid	Error message

Class Assignment

Using equivalence partition, derive test cases for the following

1. 6-digit OTP
2. Product ID from a set. Valid ids are {45, 54, 67, 76, 34}

Boundary Value Analysis (BVA)

Focuses on testing values at the edges of input ranges because defects commonly occur there. It includes:

- Minimum valid value
- The value just above minimum
- A nominal (middle) value
- Values just outside the valid range (invalid boundaries)

Boundaries are more error-prone than mid-range values.

Steps To BVA Test Generation

1. Identify the input range
2. Determine the boundary points
3. Select test values
4. Write test cases

BVA Example – Age Range 18-56

Boundary Type	Value
Below Minimum	17
Minimum	18
Just above minimum	19
Just below minimum	55
Maximum	56
Above maximum	57

BVA Test Cases

Test Case	Input	Expected Result	Note
TC1	17	Error	invalid
TC2	18	Accepted	valid
TC3	19	Accepted	Valid
TC4	55	Accepted	Valid
TC5	56	Accepted	Valid
TC6	57	Error	invalid

Manual Testing - Practical

Test Case Summary	Expected Outcome	Actual Outcome	Result

API Testing

- API stands for Application Programming Interface
- API testing is a type of testing that focuses on testing the behaviour, performance, reliability, and security of an API.
- It focuses on testing the backend instead of the front-end

Why API Testing Matters

- APIs are the backbone of modern applications. If the API application fails, the entire system fails – even if the UI look perfect.
- API testing ensures that API returns the correct data
- It ensures that API handles error gracefully
- It ensures that API performs well under load
- It ensures that the API is secure and not easily exploitable
- It ensures that the API integrates correctly with other services

What To Test In An API

Functionality – Does the API does what it was supposed to do?

- Correct HTTP status code
- Correct response body
- Correct Headers
- Correct data formats (JSON, XML)

Validation Rules

- Does it reject invalid input, such as missing fields, wrong data types, out-of-range values

What To Test In An API

Error handling: Does it return meaningful error messages?

- 400 Bad request
- 401 Unauthorized
- 404 Not Found
- 500 Internal Server Error

Performance: How fast and stable is it?

- Response time
- Throughput
- Load and stress behaviour

What To Test In An API

Security – Is the API Protected?

- Authentication – (e.g token, Oauth)
- Authorization – (role-based access)
- Input sanitization
- Rate limiting

API Testing With Postman - Practical

UI Automation Using Selenium With Java

- One of the most widely used combinations for web automation
- It is stable, fast and integrates smoothly with tools like TestNG, Maven, Gradle, and Jenkins.
- It lets you automate browsers such as Chrome, Firefox, Edge, and Safari using clean java code.

UI Automation Using Selenium With Java

It helps to automate:

Opening websites

Clicking buttons

Filling forms

Validating UI elements

Handling alerts, windows, frames

Running cross-browser tests

Building full automation framework